

Chapter 7_Finetuning

Fine-tuning is the process of adapting a model to a specific task by further training the entire model or a subset of its components.

It is most commonly used to improve the model's instruction-following ability, particularly to ensure it adheres to a specific output style and format.

Finetuning often requires more memory than what is available on a single GPU.

Transfer learning

Transfer learning focuses on how to transfer the knowledge gained from one task to accelerate learning for a new, related task. = Conceptually similar to how humans transfer skills: for example, knowing how to play the piano can make it easier to learn another musical instrument.

For example, the model transferred its knowledge of Portuguese-English and English-Spanish translation to directly translate the Portuguese to Spanish, even though there were no Portuguese-Spanish examples in the training data.

Transfer learning improves sample efficiency, allowing the model to learn the same behavior with fewer examples. A simple, efficient model learns effectively from fewer samples. For example, while training a model from scratch for legal question answering may need millions of examples, finetuning a good base model might only require a few hundred.

Alternative approach to do Finetuning Transfer learning

Feature-based transfer = a model is trained to extract features from the data, usually as embedding vectors, which are then used by another model. Part of a foundation model can be used for a classification task by adding a classifier head

It is very common for a CV.

For example, many people used a model trained on the ImageNet dataset to extract the features from images and use these features in other CV tasks such as object detection or image segmentation.

Self-supervised and Supervised

A model's training process starts with pre-training, which is usually done with self-supervision. Self-supervision allows the model to learn from a large amount of unlabeled data.

Self-supervision unlabeled data = is typically just a sequence of text without the annotation.

Self-supervision finetuning = **Continued pre-training**

With supervised finetuning, just like the autoregressor or masked model, we can finetune a model to predict the next token or fill in the blank.

The fill-in-the-blank task known as "Infilling finetuning", which is especially useful for tasks such as text editing and code debugging. You can fine-tune the model for infilling even if it was pre-trained autoregressively.

A model can also be finetune a model with reinforcement learning to generate responses that maximize human preference. Preference finetuning requires comparative data that typically follows the format (instruction, winning response, losing response)

It is possible to fine-tune a model to extend its **context length**.

Long-context finetuning typically requires modifying the model's architecture, such as adjusting the positional embedding.

A long sequence means more possible positions for tokens, and positional embeddings should be able to handle them. Compared to other finetuning techniques, the Long-context finetuning is harder to do.

When to finetune

Reasons to finetune

- To improve the model's quality
- Commonly used to generate structured outputs.
- A smaller base model is also common as it requires less memory, and, therefore, is easier to finetune --> Cheaper and faster to use in production
 - **Distillation:** A common approach is to finetune a model to imitate the behavior of a larger model using data generated by this larger model. Because this model distills the larger model's knowledge into the smaller model.

Reasons not to finetune

- Carefully crafted prompts and context can also do the same thing, like finetuning. e.g., the finetuning can help with the structured output, and so can the prompting and content
- Finetuning a model for a specific task can improve its performance for that task, but it can degrade its performance for other tasks, so this is not good for diversified prompts.

Things to consider:

- Finetuning should be the last thing to focus, it is not the first thing to prioritize. Finetuning requires high up-front investments and continual maintenance. --> need the annotated data --> slow training and expensive to acquire manually.
- Finetuning requires the knowledge of how to train the models. You need to evaluate base models to choose one to finetune, respectively, to your needs and resources.
- Once you have a finetuned model, you will need to figure out how to serve it. Like, will you host it yourself or use an API service?

So before finetuning, in AI engineering experiments should start with prompting.

- Finetuning was that it can help optimize token usage before the prompt caching was introduced, which means the more examples you add to a prompt, the more input tokens the model will use, which increases both latency and cost. Instead of including your examples in each input, you can finetune a model on examples, allowing you to use shorter prompts with a finetuned model.
 - With prompt caching, where repetitive prompt segments can be cached for reuse, this is no longer a strong benefit. Prompt caching

Finetuning and RAG

If the model fails because it lacks information or has outdated information, a RAG system that gives the model access to the relevant sources of information can help.

Information-based failures happen when the outputs are factually wrong or outdated.

On the other hand, if the model has behavioral issues, finetuning might help.

- One behavioral issue is when the model's outputs are factually correct but irrelevant to the tasks.
- When it fails to follow the expected output format

All in all, finetuning is for form, and RAG is for fact.

A RAG system gives your model external knowledge to construct more accurate and informative answers. It can also mitigate the hallucination problem.

Finetuning, on the other hand, helps your model understand and follow syntaxes and styles, and it can also reduce the hallucination problem if it is done enough with high-quality data and vice versa.

But if your model has both information and behavior issues, start with RAG.

RAG is typically easier since you will not have to worry about curating training data or hosting the finetuned models. Start with a simple term-based solution, such as TF-IDF or BM25, instead of jumping straight to the vector embedding.

- RAG can also boost the model's performance more than finetuning can
But these two are not mutually exclusive. They can sometimes be used together to maximize your application's performance.

Things to consider: There's no universal workflow for all applications.

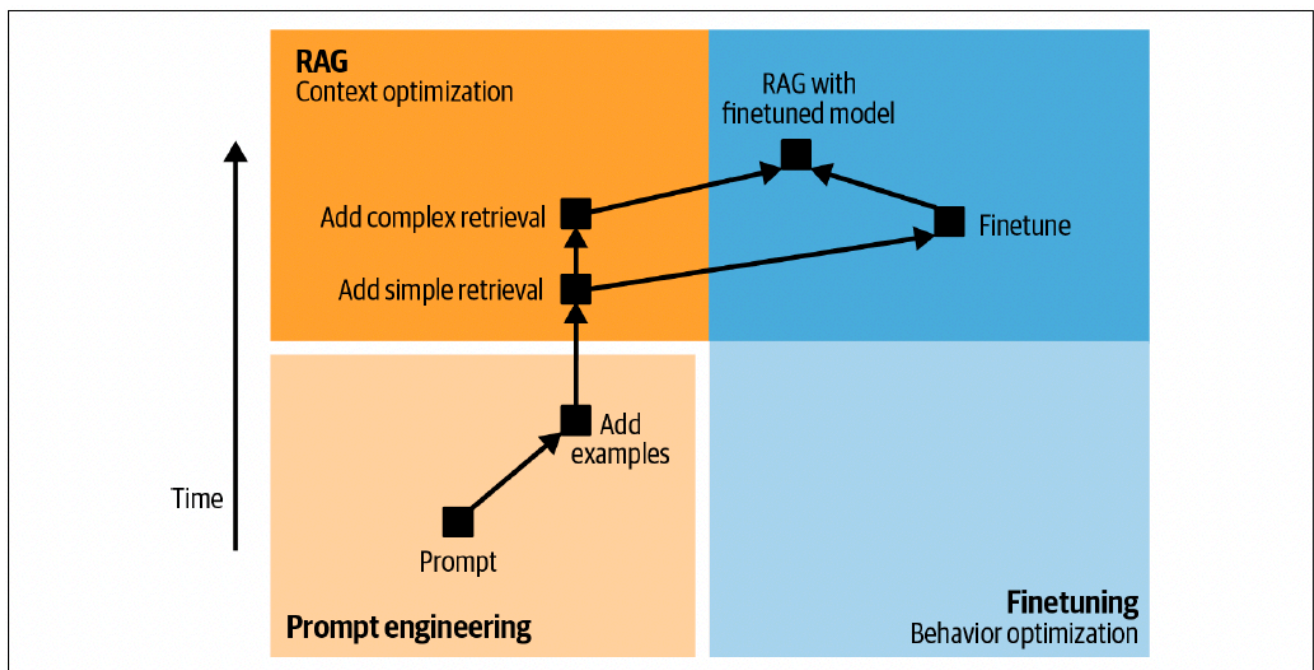


Figure 7-3. Example application development flows. After simple retrieval (such as term-based retrieval), whether to experiment with more complex retrieval (such as hybrid search) or finetuning depends on each application and its failure modes.

Memory Bottleneck

as fine-tuning is memory intensive, many fine-tuning technique aim to minimize their memory footprint. Knowing the root causes behind this memory bottleneck is essential as it could guide you to choosing the right fine-tuning technique.

and during the finetuning, GPU memory is consumed by four main categories:

1. Model weights

- FP32: 4 bytes/parameter
- FP16/BF16: 2 bytes/parameter
- A 7B model in FP16 = 14GB just for weight alone

2. Gradient

- Same size as weights (one gradient per parameter)
- Another 14GB for a 7B model in FP16

3. Optimizer states

- Adam/AdamW keeps two extra values per parameter (momentum + variance), typically stored in FP32 for stability
- That's 8 bytes/parameter just for optimizer states
- For 7B params: 56GB

4. Activations

- Memory needed to store intermediate layer outputs for the backward pass
- **Scales with:** batch size x sequence length x hidden size x number of layers
- Grows linearly with sequence length and batch size and it often the most "controllable" but also frequently underestimated

These lead to technique such as

- **LoRA (Low-Rank Adaptation)**
 - Freezes original weights, injects small trainable low-rank matrices
 - Directly combat with the optimizer state problem cause we only need gradients/optimizer states for the tiny adapter parameters (often <1% of total)
 - 7B model: adapters might be 10-50M params so optimizer states will go from 56cGB to a few hundred MB
- **QLoRA**
 - LoRA + quantizes the frozen base weights to 4-bit (NF4)
 - Solve the weights memory, a 7B model weights go from 14 GB to 3.5GB
 - Combined with LoRA's optimizer savings, this is why QLoRA can fine-tune a 7B model on a single 24GB consumer GPU
- **Gradient Checkpointing (Activation Checkpointing)**
 - Attacks the activations bottleneck specifically
 - Trades compute for memory: discards activations, recomputes them during backward pass
 - Roughly memory reduction for n layers, at cost of 20-30% more compute time
 - Orthogonal to LoRA/QLoRA, and they are often stacked together
- **Gradient Accumulation**
 - Doesn't reduce memory for a single forward/backward pass, but lets you simulate large batch sizes without the activation memory of actually processing a large batch at once
 - solve the "need bigger batch size but can't afford activation memory" problem
- **8-bit Optimizer (bitsandbytes)**
 - Attacks optimizer states directly without changing the training method
 - put the Adam states in 8-bit instead of 32-bit , leading 4x reduction in that specific component
- **Model/tensor parallelism, FSDP/DeepSpeed ZeRO** are all used for the multiple GPUs, model does not fit on one.
 - **ZeRO-1:** shards optimizer states across GPUs
 - **ZeRO-2:** shards optimizer states + gradients
 - **ZeRO-3:** shards optimizer states + gradients + weights

And each stage trying to solve the memory problems above, but the there are more of communication needed for the hardware to communicate.

Other Memory consumer

- **Backpropagation:** Forward pass and backward pass have fundamentally different memory profiles. and a key factor that determines a model's memory footprint during finetuning is its number of **trainable parameters**.

Note: The **trainable parameter** is a parameter that can be updated during the finetuning.

- **Forward pass:**

Input → Layer1 → Layer2 → ... → LayerN → Loss

Each layer produces an activation (its output). During the forward pass, these activations must be kept in memory because the backpropagation needs them.

- **Backward pass:**

Loss → dLayerN → ... → dLayer2 → dLayer1

So to compute the gradient at Layer_k, we need

- The gradient flowing back from Layer_k+1
- The **activation that was stored during the forward pass at Layer_k** (used in the chain rule, e.g., derivative of ReLU needs to know the input value)
 - This is why storing the activation is essential after the forward pass because they are needed again during the backward pass. and this interaction/dependency is exactly what the Gradient Checkpointing comes in hand (instead of storing the every activation at every layer, store only some (e.g., every 5th layer) and recompute the missing one on the fly during the backward pass by rerunning the forward pass in those segments.)

- **Inference Memory and Training Memory:**

- **Inference (Forward pass only, no learning)**

Inference_Memory = Weights + Activations (transient, small) + KV cache (for generation)

as we can see here, there is no gradient, optimizer states, and the activations are freed immediately after use (no need to retrain for backward pass)

Thus, for 7 B model, FP16 inference: 14GB weights + a few GB for KV cache/activations depending on context length and batch size. --> total around 16 to 20GB

- **Training (Forward + backward + Optimizer update)**

Memory = Weights + Activation (has to be retained from the Backpropagation) + Gradients + Optimizer States

So that means that the 7B model, full fine-tuning, FP16/mixed precision: the final memory would be around 100 - 120GB

So if we break this down into the ratio: for a 7B model

- Inference account for 16 - 20GB
 - Full fine-tuning training = 100 - 120GB
 - LoRA Training = 18 - 24GB
 - it is noticeable that the inference and the LoRA memory is relatively close. and that is the whole point because we only have to account for the "load the model" cost, plus a small tax 4 - 5GB for the adapters and the activation.

Note: Optimizer state holds the algorithm's tracking variables like the.

historical gradients and momentum. For the algorithms like Adam, this state requires roughly twice the memory of the model parameter themselves because the optimizer states are all kept in high precision even when everything (weights, gradients,...) is compressed to minimize the memory usage.

Quantization

It is a technique for reducing the precision of numbers used to present a model's weights and activations (i.e., going from FP32 down to lower-precision formats like FP16/BF16)

Goal: to make models smaller, faster, and cheaper to run while trying to preserve as much accuracy as possible

Note: NN weights does not need the full dynamic range and precision of FP32 to make good predictions.

What to quantize:

Quantize whatever is consuming most of your memory without hurting the performance.

Weights quantization is more common than activation quantization, since weight activation tends to have stable impact on performance with less accuracy loss.

When to quantize:

It can happen during the **training** or **post-training**.

- **Post-training Quantization:** quantizing the model after it has been fully trained. (Most common), using a small calibration dataset to determine good scale factors. fast and easy, but lose more accuracy, especially at very low bit-width.

- **Training Quantization:** it is not as much as popular as the PTQ but it has two main goals:

1. To produce a model that can perform well in low precision during inference. This is to address the **challenge that a model's quality might degrade during the post-training quantization**.

2. To reduce training time and cost. Quantization reduces a model's memory footprint, allowing the model to be trained on cheaper hardware or allowing the training of a larger model on the same hardware. Quantization also speeds up computation, which further reduces costs.

- **Quantization-Aware Training (QAT):** Simulate quantization effects during the training (using "fake quantization" nodes) so the model learns weights that are robust to the precision loss. More accurate but requires retraining.

- **Weight-only and the Weight + Activation quantization:** Sometimes only the weights are quantized (activations stay in high precision), which is common for LLM inference since activations can have outlier that are harder to quantize well.

Granularity

- **Per-tensor:** One scale factor for an entire weight matrix
- **Per-channel:** a separate scale factor per output channel/row - more accurate, still cheap.
- **Per-group / block-wise:** scale factors for small chunks of a tensor (e.g., every 32 or 64 values), used in modern LLM quantization methods (like GPTQ, AWQ) to handle outliers better at very low bit-widths.

It is good for the hardware that has fast INT8/INT4 kernels, **lower power consumption and faster inference (computation speed)** if the quantization is used to reduce the memory footprint.

But it will cause **potential accuracy degradation**, especially below 8 bits, and outlier values (common in large transformer models) can cause big errors if not handled carefully which is why modern LLM quantization methods spend a lot of effort on outlier-aware or mixed-precision schemes. and each conversion often causes a small value change, and many small changes can cause a big performance change.

- You cannot have fewer than 1 bit per value.

Standard: Inference in the low precision is now the criterion, so a model is trained using a higher precision format to maximize performance, then its precision is reduced for inference.

Finetuning Technique

Full FineTuning, the number of trainable parameters is exactly the same as the number of the parameters, making it memory-expensive. It can look similar to training, but the main difference is that training starts with randomized model weights, whereas finetuning starts with models weights that have previously trained.

Parameter-Efficient FineTuning (PEFT)

This method aims to get performance close to the full finetuning while updating only a tiny fraction of the model's parameters.

Upside

- **Dramatically lower memory requirement**
- **Lower compute cost and faster training:** fewer gradient computations to update, speeding up each training step and reduce the cost of experimentation.
- **Smaller artifact to store and ship:** the adapter weights = megabytes
- **Easier multi-tasking/Multi-tenant serving:** Because adapters are small and modular, you can keep one frozen base model in memory and **hot-swap** different LoRA adapters per request or per customer, which present much more efficiency than hosting separate fully-finetuned models for each use case.
- **Less catastrophic forgetting:** since the original weights are frozen, the model's general capabilities are more likely preserved. Training separate small adapters per task avoid the risk of one task overwriting another's knowledge.

- **Comparable performance to full finetuning in many cases**

Downside

- **Performance ceiling:** for tasks that require deep changes to the model's underlying knowledge or reasoning.
- **Still does not add new facts well:** with few trainable parameters,
- **Extra architectural complexity:** have to choose where to insert adapters/LoRA matrices (which layers, which rank), which adds a design/hyperparameter surface that full finetuning does not have. and getting this wrong can leave meaningful performance on the table.
- **Inference-time overhead:** depending on implementation, adapters can add a small amount of latency at inference if they are not merged into the base weights. and the LoRA can be merged back into the weights to avoid this, but we have to sacrifice the adapter swappable benefit
- **Still requires a correctly frozen, high quality base model:** it assumes the frozen base model already has most of the needed capability, so if the base model is a poor fit for the task domain, a small adapter often can not compensate as well as retraining more of the network could.
- **Data quality sensitivity remains:** so a small number of trainable parameters trained on inequality data can still produce a poorly-behaved model and nobody wants that.

PEFT Technique

- **Adapter-based methods:** Small trainable modules (adapters) are inserted into the model's architecture while the original weights stay frozen.
- **LoRA (Low-Rank Adaptation):** Instead of updating the full weight matrices directly, we freeze the entire and train small low-rank decomposition matrices, drastically cutting the number of trainable parameters while approximating full finetuning quality.

- (frozen, original weights) shared

- where W is $n \times m$ and W_0 is $n \times m$, with (the "rank") being much smaller than n or m

- So during the training, only W_A and W_B are updated. then the output becomes $W_0 + W_A W_B$ and because the rank r is small (often 4, 8, 16, or 64), and together have far fewer parameters than W or sometimes less than 1% of the original layer's parameter count.

The intuition is that the change needed to adapt a pretrained model to a new task tends to have a low "intrinsic rank", meaning that it does not require the full expressive power of a dense weight update. A low-rank approximation captures most of the useful adaptation signal.

- LoRA is typically applied to the **attention layers** (**query/key/value/output projection matrices**) in the transformer block rather than every layer in the network, since that tends to give a good tradeoff between **performance** and **efficiency**, though it can be applied more broadly too.

- **Benefits:**
 1. **Memory efficiency:** we do not need to store gradient/optimizer states for the full weight matrix, only for the tiny A/B matrices.
 2. **Storage efficiency:** so instead of saving a full copy of the model per task, you save just the small LoRA weights (often just a few MB), and swap them in on top of the shared base model.
 3. **No inference latency cost:** after the training, can be merged directly into , so at inference time there is no extra computation compared to the original model.
 4. **Multiple adapters:** you can keep several LoRA modules for different tasks/behaviors and switch between them without duplication the whole model.
- **QLoRA (Quantized LoRA):** a variants of the LoRA, combine LoRA with quantization (loading the frozen base model in 4-bit precision, shrinking it from 16-bit or 32-bit precision), while keeping the trainable LoRA adapters () in higher precision (typically in 16-bit/bfloat16) to cut memory needs even further, making it possible to fine-tune large models on a single consumer GPU.
- The **trainable parameter** in this finetuning technique account for only 1% of the whole parameter, and besides the trainable parameter, all parameter are frozen, not trainable, but the Back propagation can flow through that to compute the gradients where needed, but the optimizer only updates the small trainable parameter. This help alleviate the memory-expensive bottleneck, because with more trainable parameter:
 - The gradient for each trainable parameter
 - The optimizer state for each trainable parameter (e.g., Adam keeps two extra numbers per parameter: **momentum and variance**)

Remember: The more trainable parameters (get updated during the training, aka, [Back propagation](#) in the direction that reduces the loss) there are, the more memory is needed.

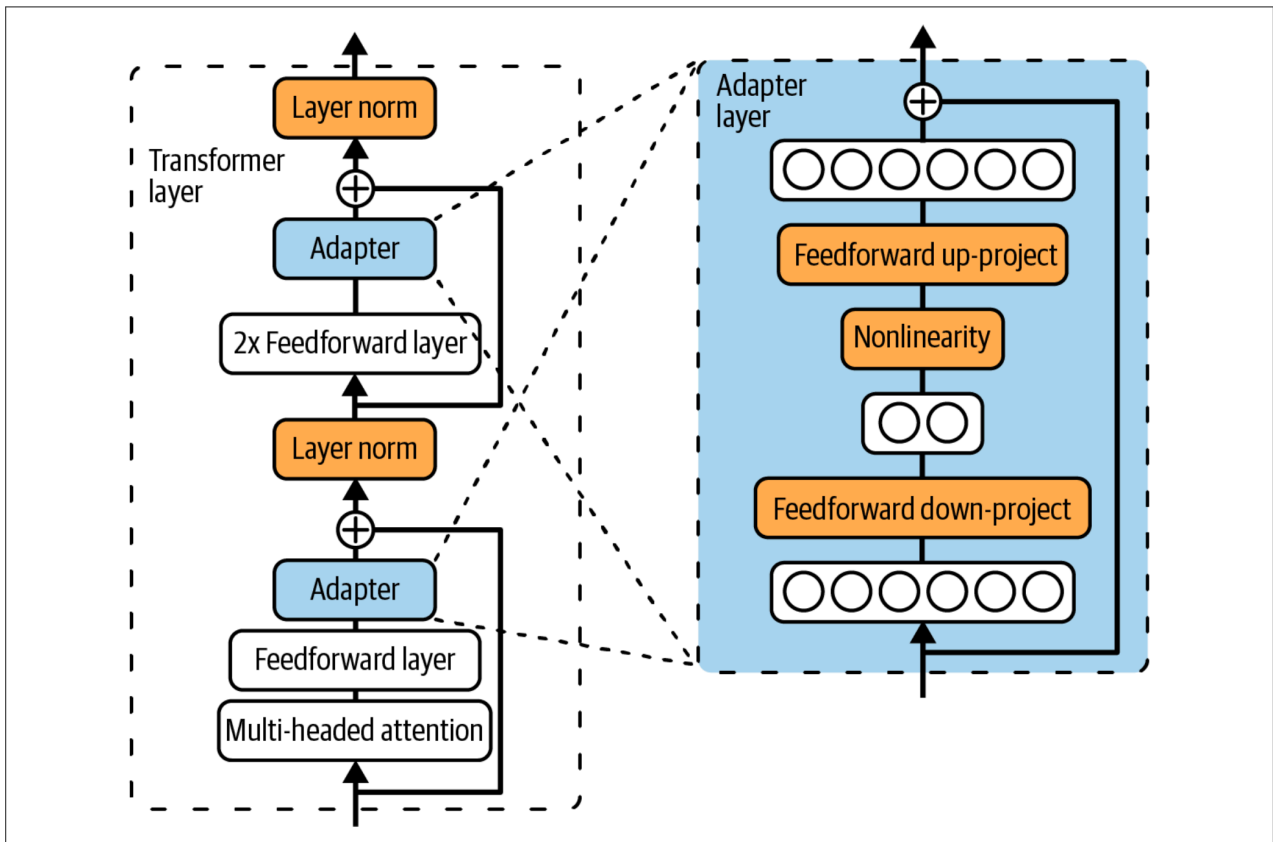


Figure 7-8. By inserting two adapter modules into each transformer layer for a BERT model and updating only the adapters, Houlsby et al. (2019) were able to achieve strong finetuning performance using a small number of trainable parameters.

Model Merging and Multi-Task FineTuning

Strategies for getting a single model (or a system) to handle multiple tasks well.

The Multi-Task FineTuning

It means we fine-tune one model on a mixture of tasks simultaneously, rather than training separate models per task.

Process

- gathering training data for multiple tasks (e.g., summarization, translation, classification, code generation)
- combine them into one training mixture, often with task-specific instructions or prompt to signal which task is being performed
- train a single model on this combined dataset, updating all (or LoRA-adapted) parameters jointly.

and then

- The tasks often share underlying skills (reasoning, language and understanding), so training on many tasks together can improve generalization, this is a part of why instruction-tuned models generalize to unseen tasks better than single-task models. and we will get one deployable model instead of many, simplifying serving infrastructure.

Challenges:

- **Task interference/negative transfer:** tasks can compete for model capacity, and gains on one task can come at the cost of another if they pull the model in conflicting directions.
- **Data balancing:** if one task has far more data than others, the model can become skewed toward it, careful sampling/weighting is needed
- **Catastrophic forgetting:** if we fine-tune sequentially rather than jointly, earlier tasks can degrade as new ones are learned.
- and the retraining is needed if you want to add or adjust a task later, unless you design for incremental updates.

Model Merging

it is a popular alternative, and instead of training one model jointly on many tasks, you train separate specialized models (or adapters) per task independently, then combine their weights directly, without further training to get a model that performs well across all of them.

Goal: simply put, it is used to create a single model that provides more value than using all the constituent models separately.

Merging technique:

- **Linear/Weighted averaging:** average the weights of multiple fine-tuned and base model weights (this often works if the models are fine-tuned from the same base checkpoint)
- **Task arithmetic:** compute a "task vector" (the difference between the fine-tuned and base model weights) for each task, then add/subtract these vectors to combine or even remove capabilities.
- **SLERP (Spherical Linear interpolation):** interpolate between model weights along a spherical path rather than linearly, often preserving more of each model's characteristics.
- TIES-merging, DARE,
- Summing
- Layer stacking: take different layers from one or more models and stack them on top of each other
- Concatenation

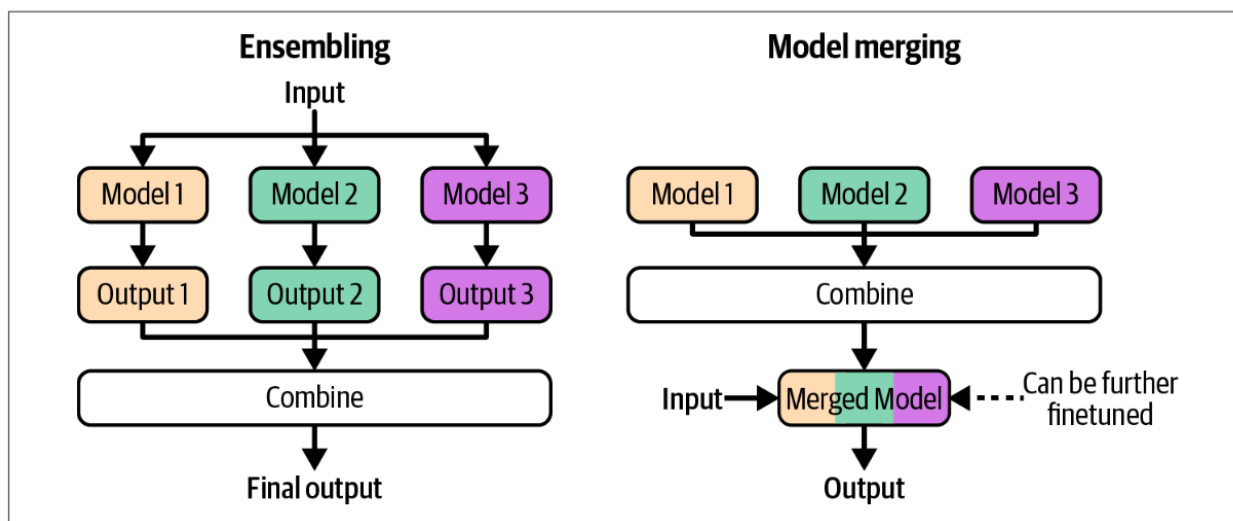


Figure 7-13. How ensembling and model merging work.

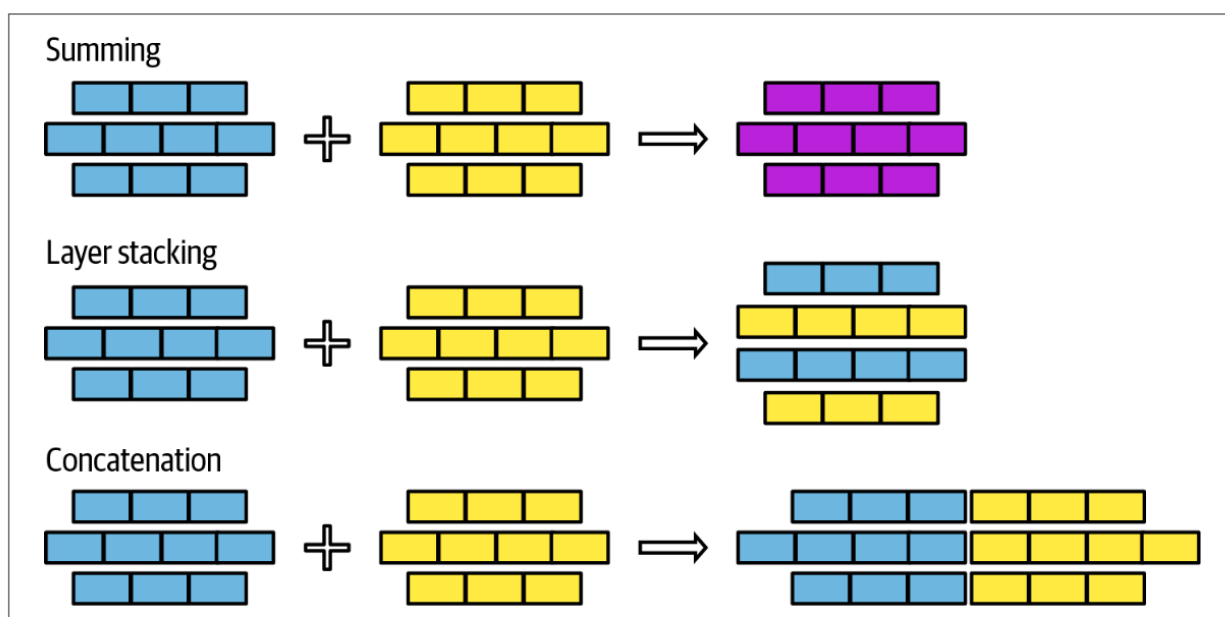


Figure 7-14. Three main approaches to model merging: summing, layer stacking, and concatenation.

And one key benefit that make the merging shine, since LoRA adapters are small and modular, merging is easy, we can combine multiple LoRA adapters (each trained for a different task) into a single set of weights, or even keep them separate and switch/combine them dynamically at inference time.

Challenges

- Merging quality depend heavily on how similar/related the tasks and base models are, so merging very divergent fine-tunes can degrade performance on all tasks
- It is more of an empirical, trial-and-error art than a principled guarantee
- Debugging is harder: If a merged model underperforms, it is not always clear which component or interaction is responsible.

Finetuning Tactics

Finetuning frameworks

- LLaMa-Factory
- Unsloth
- PEFT
- Axolotl
- LitGPT

Finetuning Hyperparameter

- Learning rate
- Batch size
- Prompt loss weight
- Number of epoch: an epoch is a pass over the training data. The number of epochs determines how many times each training example is trained on.